

Computer Architecture

(Instructions: Part I)

Dept. Applied AI, SeoulTech

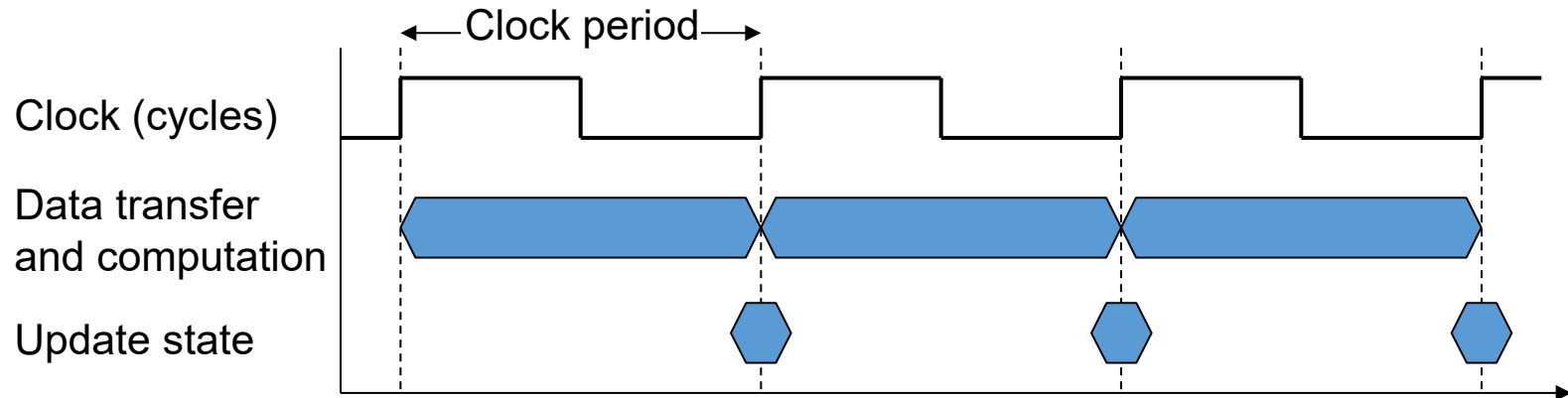
Beomseok Oh

Contents

- Instructions: Language of the computer
- Logical operations
- Representing instructions in the computer
- Instructions for making decisions

(Recall) CPU Clocking

- Operation of digital hardware governed by a constant-rate clock



- Performance improved by
 - Reducing number of clock cycles
 - Increasing clock rate
 - Hardware designer must often trade off clock rate against cycle count

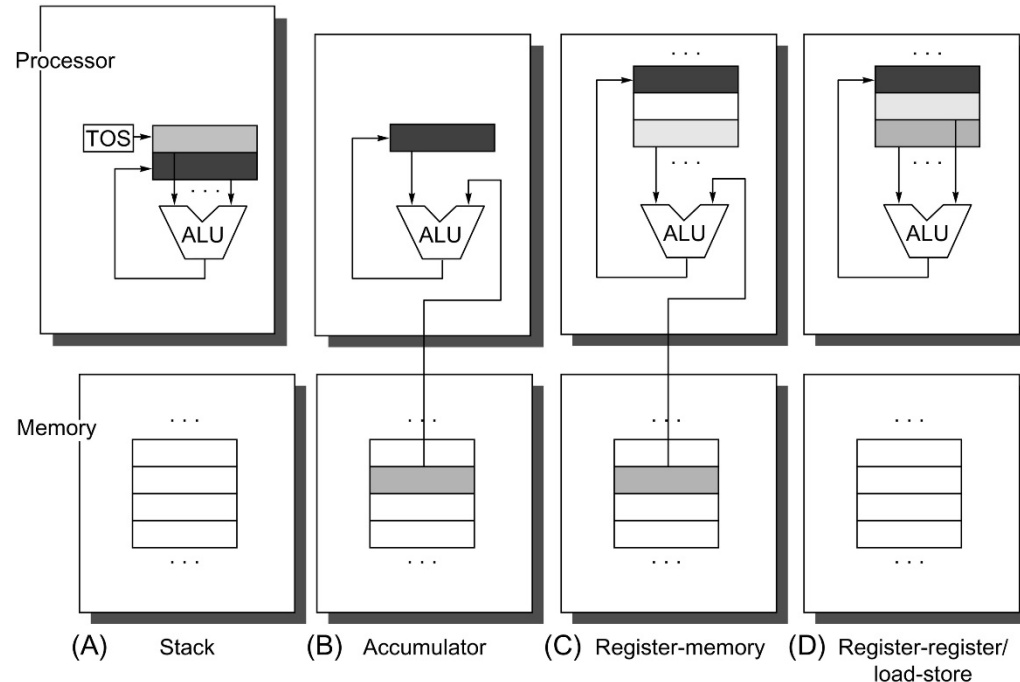
$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

(Recall) Classifying instruction set archit.

- A register is a CPU component specifically designed to store a **small amount of data**
- Each register stores 32 bits or 64 bits data depending on the word size (32 or 64 bits)
 - In this lecture, we will consider **RV32 only** (which uses 32-bit registers)
- RISC-V base ISA consists of **32 general-purpose registers (GPR)** x1-x31 that hold integer values and x0 is reserved to hold zero

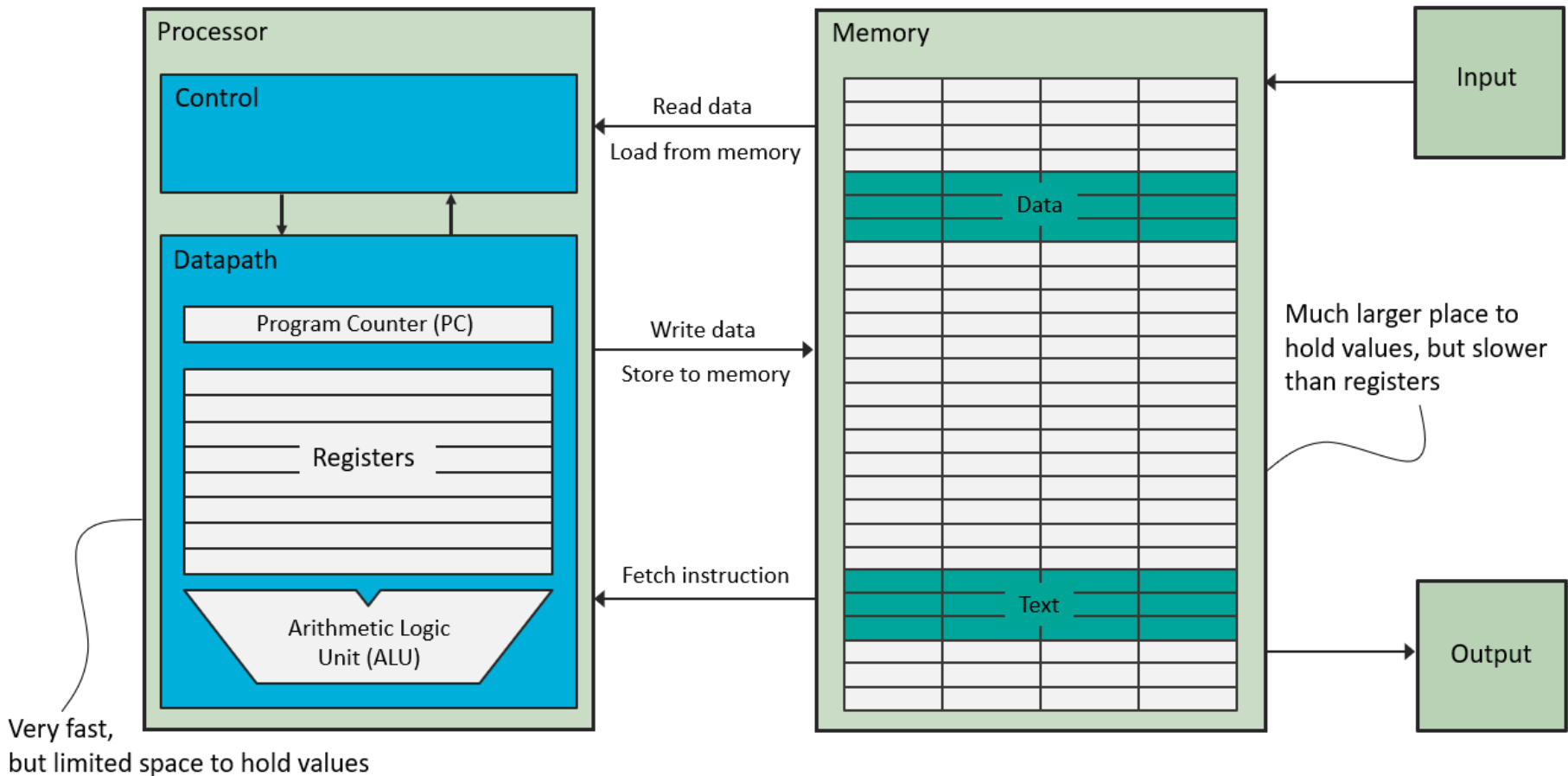
(Recall) Classifying instruction set archit.

- RISC-V is a **load-store architecture**

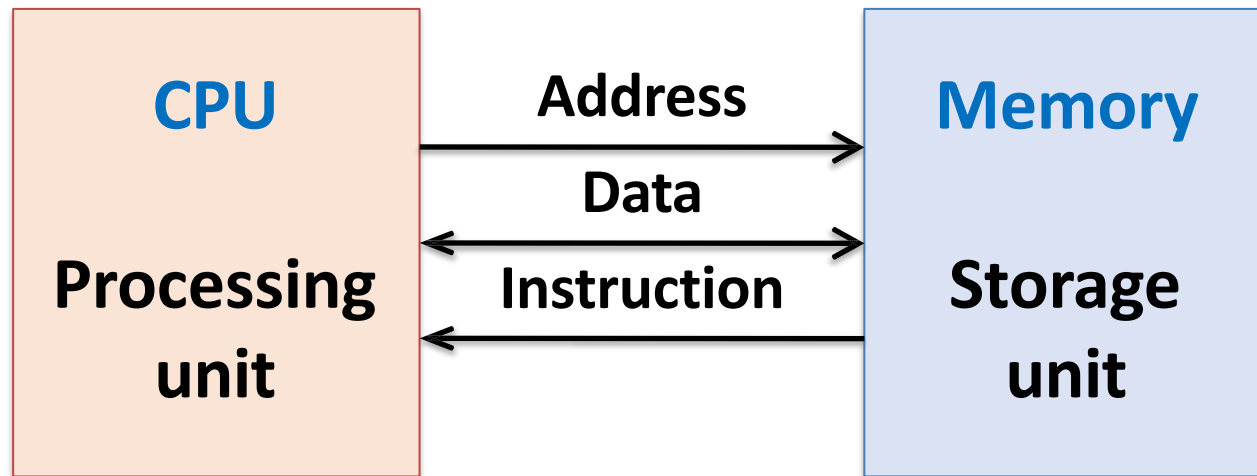


Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1, A	Load R1, A
Push B	Add B	Add R3, R1, B	Load R2, B
Add	Store C	Store R3, C	Add R3, R1, R2
Pop C			Store R3, C

(Recall) Register (Load-store)



(Recall) CPU and memory



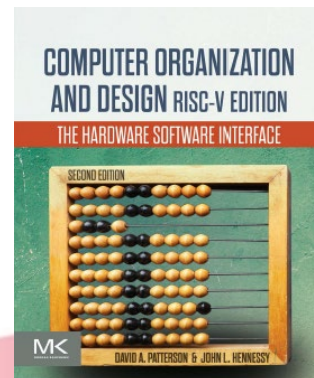
Data movement
Arithmetic & logical ops
Control transfer

Byte addressable array
Code + data (user program, OS)
Stack to support procedures

Instructions:

Language of the Computer

(ch. 2, COD book)



RISC-V instruction: overview

- Instruction format
 - Each line of RISC-V assembly code is a **single instruction**
 - RISC-V arithmetic, logical, shift and branch instructions must always have **three operands**
 - Two sources and one destination
 - RISC-V data transfer, unconditional branch instructions have **two operands**
 - One source and one destination
- RISC-V instructions are generally written in the format
 - <instruction> <destination register> <operands>
 - `add x5, x10, x6`
 - `lw x7, 4(x6)`
 - Operands are registers or memory
- *Design Principle 1: Simplicity favors regularity*
 - **Regularity** makes implementation simpler
 - **Simplicity** enables higher performance at lower cost

RISC-V instruction: overv

```
void sort(int v[], int n) {
    for (int i = 1; i < n; ++i) {
        temp = v[i];
        for(int j=i-1; j>=0 && v[j]>=temp; --j)
        {
            v[j+1] = v[j];
        }
        v[j+1] = temp;
    }
}
```

Compiler
→

Sort:

```
addi sp, sp, -12    # make room on stack
sw   ra, 8(sp)      # save return address regs.
sw   s1, 4(sp)      # save saved register s1
sw   s2, 0(sp)      # save saved register s2
addi t0, a0, 0      # t0 has *v
addi t1, a1, 0      # t1 has n
addi t2, x0, 1      # t2 has i=1
```

Loop1:

```
bge t2, t1, Exit    # if (i >= n) goto Exit
slli t4, t2, 2       # t4 has offset i * 4
add s1, t0, t4       # s1 has address of v[i]
lw  t4, 0(s1)        # t4 has v[i]
addi t3, t2, -1      # t3 has j=i-1
```

Loop2:

```
blt t3, x0, Loop3    # if (j < 0) goto Loop3
slli t5, t3, 2       # t5 has offset j * 4
add s2, t0, t5       # s2 has address of v[j]
lw  t5, 0(s2)        # t5 has v[j]
blt t5, t4, Loop3    # if (v[j] < temp) goto L3
sw  t5, 4(s2)        # v[j+1] = v[j]
addi t3, t3, -1      # j--
jal x0, Loop2        # goto Loop2
```

Loop3:

```
slli t5, t3, 2       # t5 has offset j * 4
add s2, t0, t5       # s2 has address of v[j]
sw  t4, 4(s2)        # t4 has v[j+1]
addi t2, t2, 1       # i++
jal x0, Loop1        # goto Loop1
```

Exit:

```
lw s2, 0(sp)         # restore saved register s2
lw s1, 4(sp)         # restore saved register s1
lw ra, 8(sp)         # restore return address
addi sp, sp, 12      # restore stack pointer
jalr x0, 0(ra)       # return to caller
```

RISC-V instruction: overview – Cont'd

RISC-V assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	Add	add x5, x6, x7	$x5 = x6 + x7$	Three register operands; add
	Subtract	sub x5, x6, x7	$x5 = x6 - x7$	Three register operands; subtract
	Add immediate	addi x5, x6, 20	$x5 = x6 + 20$	Used to add constants
Data transfer	Load word	lw x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Word from memory to register
	Load word, unsigned	lwu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Unsigned word from memory to register
	Store word	sw x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Word from register to memory
	Load halfword	lh x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Halfword from memory to register
	Load halfword, unsigned	lhu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Unsigned halfword from memory to register
	Store halfword	sh x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Halfword from register to memory
	Load byte	lb x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Byte from memory to register
	Load byte, unsigned	lbu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Byte unsigned from memory to register
	Store byte	sb x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Byte from register to memory
	Load reserved	lr.d x5, (x6)	$x5 = \text{Memory}[x6]$	Load; 1st half of atomic swap
	Store conditional	sc.d x7, x5, (x6)	$\text{Memory}[x6] = x5; x7 = 0/1$	Store; 2nd half of atomic swap
	Load upper immediate	lui x5, 0x12345	$x5 = 0x12345000$	Loads 20-bit constant shifted left 12 bits
Logical	And	and x5, x6, x7	$x5 = x6 \& x7$	Three reg. operands; bit-by-bit AND
	Inclusive or	or x5, x6, x8	$x5 = x6 x8$	Three reg. operands; bit-by-bit OR
	Exclusive or	xor x5, x6, x9	$x5 = x6 \wedge x9$	Three reg. operands; bit-by-bit XOR
	And immediate	andi x5, x6, 20	$x5 = x6 \& 20$	Bit-by-bit AND reg. with constant
	Inclusive or immediate	ori x5, x6, 20	$x5 = x6 20$	Bit-by-bit OR reg. with constant
	Exclusive or immediate	xori x5, x6, 20	$x5 = x6 \wedge 20$	Bit-by-bit XOR reg. with constant
Shift	Shift left logical	sll x5, x6, x7	$x5 = x6 \ll x7$	Shift left by register
	Shift right logical	srl x5, x6, x7	$x5 = x6 \gg x7$	Shift right by register
	Shift right arithmetic	sra x5, x6, x7	$x5 = x6 \gg x7$	Arithmetic shift right by register
	Shift left logical immediate	slli x5, x6, 3	$x5 = x6 \ll 3$	Shift left by immediate
	Shift right logical immediate	srl_i x5, x6, 3	$x5 = x6 \gg 3$	Shift right by immediate
	Shift right arithmetic immediate	srai x5, x6, 3	$x5 = x6 \gg 3$	Arithmetic shift right by immediate
Conditional branch	Branch if equal	beq x5, x6, 100	if ($x5 == x6$) go to PC+100	PC-relative branch if registers equal
	Branch if not equal	bne x5, x6, 100	if ($x5 != x6$) go to PC+100	PC-relative branch if registers not equal
	Branch if less than	blt x5, x6, 100	if ($x5 < x6$) go to PC+100	PC-relative branch if registers less
	Branch if greater or equal	bge x5, x6, 100	if ($x5 \geq x6$) go to PC+100	PC-relative branch if registers greater or equal
	Branch if less, unsigned	bltu x5, x6, 100	if ($x5 < x6$) go to PC+100	PC-relative branch if registers less, unsigned
	Branch if greater or equal, unsigned	bgeu x5, x6, 100	if ($x5 \geq x6$) go to PC+100	PC-relative branch if registers greater or equal, unsigned
Unconditional branch	Jump and link	jal x1, 100	$x1 = \text{PC}+4$; go to PC+100	PC-relative procedure call
	Jump and link register	jalr x1, 100(x5)	$x1 = \text{PC}+4$; go to $x5+100$	Procedure return; indirect call

RISC-V instruction: Comments

- Comments are written using the # symbol
 - Anything written after a # on a line gets ignored (like Python)
- Comments are far more important in assembly language than in other languages
 - RISC-V instructions consists of register number or memory address
 - We do not have variable names at all!

RISC-V arithmetic instruction

- RISC-V arithmetic instruction must always have **three operands**
 - Two sources and one destination
- Consider an example
 - $a = b + c$ (in C)
 - `add a, b, c` (in RISC-V)
- Suppose that **RISC-V registers** `x5`, `x6`, `x7` are associated with C variables `a`, `b`, `c`. Then, the above RISC-V instruction is written
 - `add x5, x6, x7`
- Subtraction operation
 - $a = b - c$ (in C)
 - `sub x5, x6, x7` (in RISC-V)
- **All arithmetic operations have this format**

RISC-V arithmetic instruction – Cont'd

- What if we have **more than two** source operand variables?

- $a = b + c + d + e;$

- `add a, b, c`

- `add a, a, d`

- `add a, a, e`

- $f = (g + h) - (i + j);$

- `add t0, g, h`

- `add t1, i, j`

- `sub f, t0, t1`

- Using the registers ($a, b, \dots, e$ in $x5, x6, \dots, x9$):

- `add x5, x6, x7`

- `add x5, x5, x8`

- `add x5, x5, x9`

- `add x5, x20, x21`

- `add x6, x22, x23`

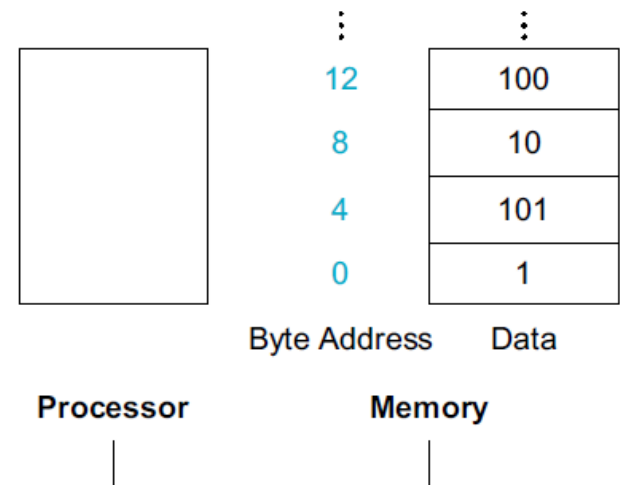
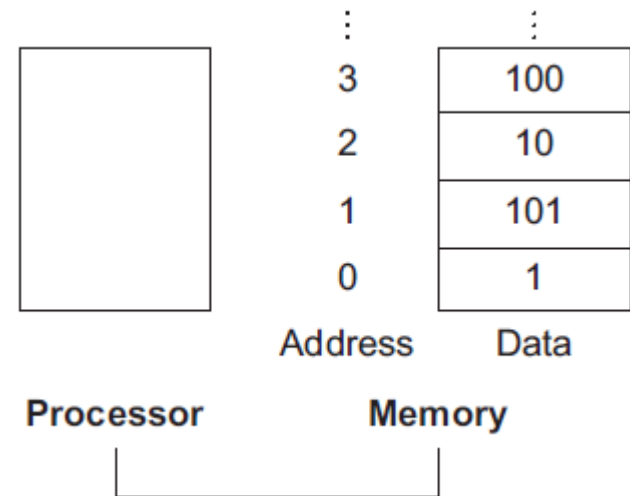
- `sub x19, x5, x6`

Register operands

- Arithmetic instructions **use register operands**
- RISC-V has a 32×32 -bit register file
 - Use for **frequently accessed data**
 - 32-bit data is called a “**word**”
 - 64-bit data is called a “**doubleword**”
 - 32 x 64-bit general purpose registers x0 to x31
- *Design Principle 2: Smaller is faster*
 - c.f. main memory: millions of locations
 - Why?

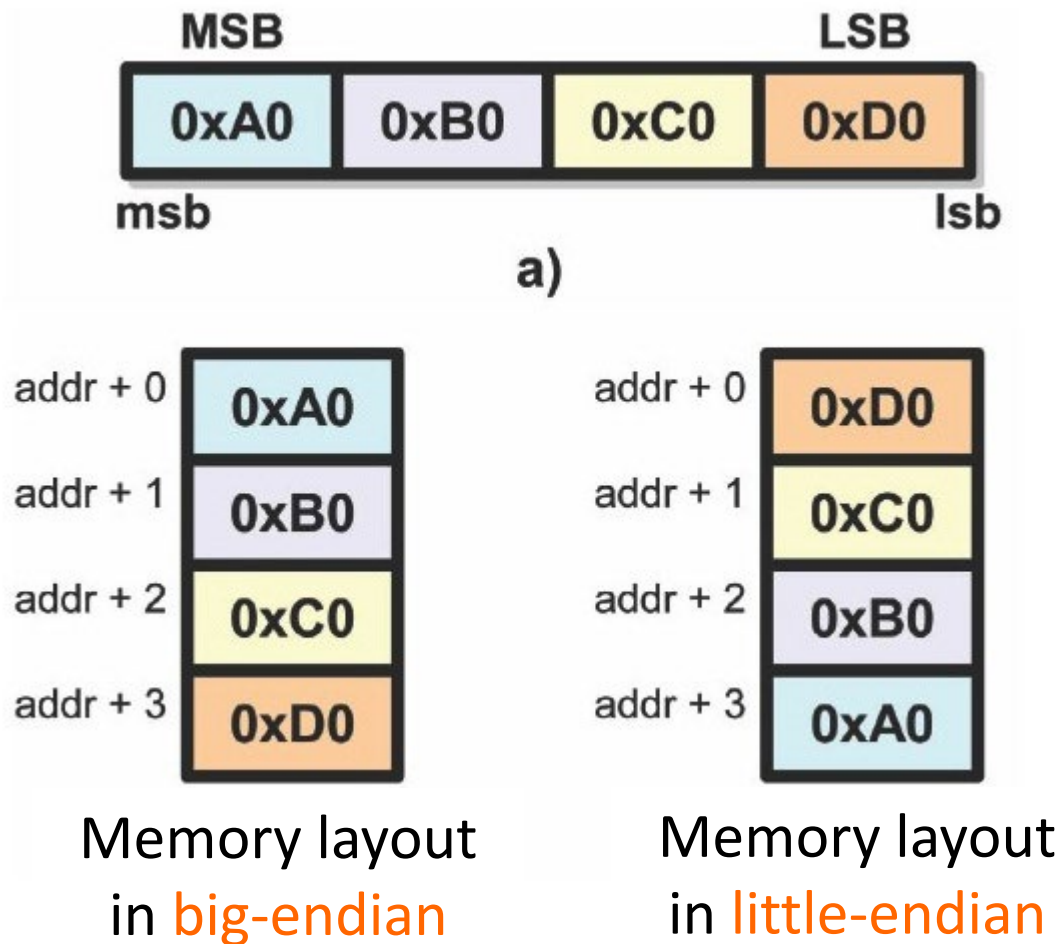
Memory operands

- Main memory is used for **composite data**
 - Arrays, structures, dynamic data
- To apply arithmetic operations
 - **Load** values from memory into registers
 - **Store** result from register to memory
- Memory is **byte addressed**
 - Each address identifies an 8-bit byte
- RISC-V is **little endian**
 - Least-significant byte is located at least address of a word
 - *c.f.* Big Endian: most-significant byte is located at least address
- RISC-V does not require words to be aligned in memory
 - Unlike some other ISAs



Memory operands – Cont'd

- Big-endian vs. little-endian

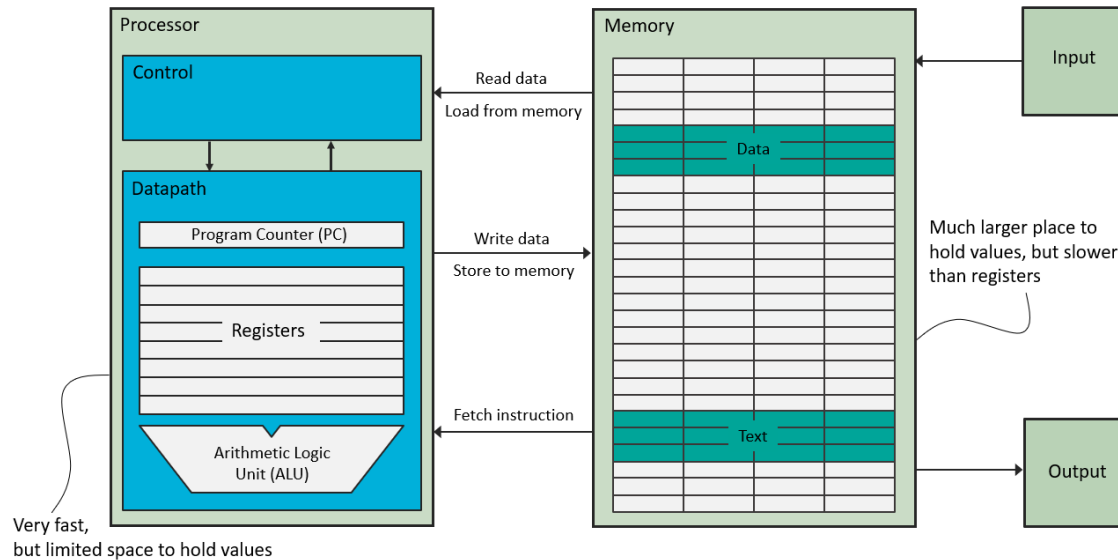


Memory operands – Cont'd

- Consider an example C code:
 - `A[12] = h + A[8];`
 - `h` in `x21`, base address of `A` in `x22`
- Compiled RISC-V code:
 - `ld x9, 64(x22)`
 - `add x9, x21, x9`
 - `sd x9, 96(x22)`
- `A` and `8` of `A[8]` are respectively called the **base register** and the **offset**

Registers vs. Memory

- Registers are faster to access than memory
 - Assuming 32-bit data, registers are roughly **200 times faster** and are **10,000 times more energy efficient** than DRAM in 2020
- Operating on memory data requires loads and stores
 - **More instructions to be executed!**



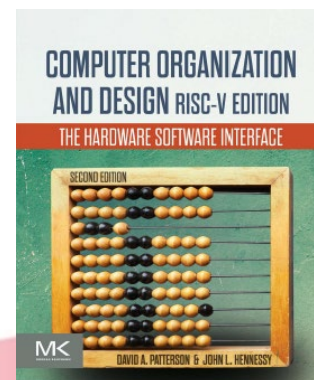
- Compiler must use registers for variables as much as possible
 - Only spill to memory for less frequently used variables
 - Register **optimization** is important!

Constant or Immediate Operands

- Many times a program use a constant in an operation
 - E.g., incrementing an index to point to the next element of an array
- Constant data specified in an instruction
 - Option 1
 - `lw x9, AddrConstant4(x3)`
 - `add x22, x22, x9`
 - Option 2
 - `addi x22, x22, 4`
- Make the **common case fast**
 - Small constants are common
 - Immediate operand **avoids a load instruction**
- Register `x0` to be **hard-wired** to the **value zero**

Logical operations

(ch. 2.5, COD book)



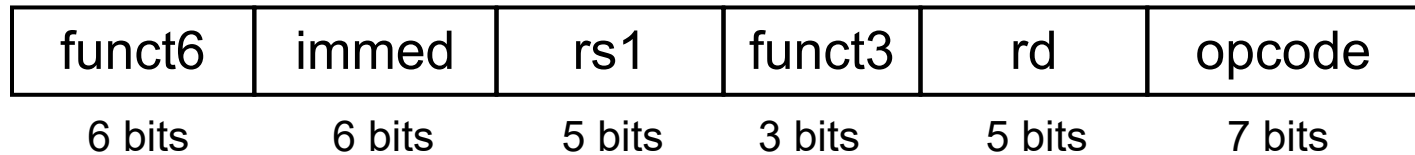
Logical operations

- Instructions for bitwise manipulation

Logical operations	C operators	Java operators	RISC-V instructions
Shift left	<<	<<	sll, slli
Shift right	>>	>>>	srl, srli
Shift right arithmetic	>>	>>	sra, srai
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit XOR	^	^	xor, xori
Bit-by-bit NOT	~	~	xori

- Useful for extracting and inserting groups of bits in a word

Shift operations



- immed: how many positions to shift

- Shift left logical

- Shift left and fill with 0 bits
- `slli` by i bits multiplies by 2^i
- Example

00000000 00000000 00000000 00001001 = 9

Shift left by 4 bits results

00000000 00000000 00000000 10010000 = 144

- Shift right logical

- Shift right and fill with 0 bits
- `srl` by i bits divides by 2^i (unsigned only)

AND operation

- Useful to mask bits in a word
 - Select some bits, clear others to 0

and x9, x10, x11

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00001100 00000000

OR operation

- Useful to include bits in a word
 - Set some bits to 1, leave others unchanged

or x9, x10, x11

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00111101 11000000

XOR operation

- Differencing operation

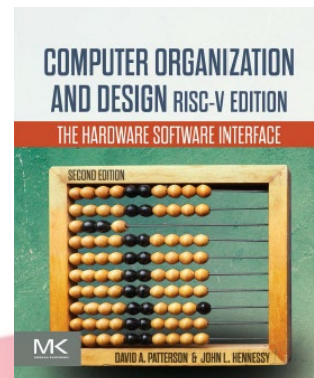
- Set some bits to 1, leave others unchanged

`xor x9, x10, x12`

x10	00000000	00000000	00000000	00000000	00000000	00000000	00001101	11000000
x11	11111111	11111111	11111111	11111111	11111111	11111111	11111111	11111111
x9	11111111	11111111	11111111	11111111	11111111	11111111	11110010	00111111

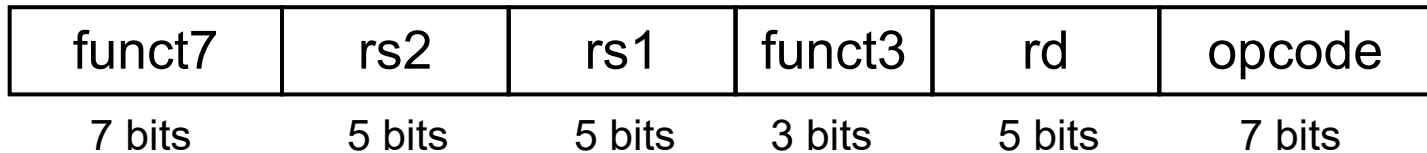
Representing instructions in the computer

(ch. 2.5, COD book)



RISC-V instruction format

- RISC-V instructions are represented as **binary numbers**, just like data
- In keeping with the design principle of simplicity favors regularity, **RISC-V instructions are all 32 bits long**



- The layout of the instruction is called the **instruction format**
- Instruction fields (**R-type format**)
 - opcode: operation code
 - rd: destination register number
 - funct3: 3-bit function code (additional opcode)
 - rs1: the first source register number
 - rs2: the second source register number
 - funct7: 7-bit function code (additional opcode)

RISC-V instruction format – Cont'd

- Example: add instruction

- add x9, x20, x21

0000000	10101	10100	000	01001	0110011
0	21	20	0	9	51
funct7	rs2	rs1	funct3	rd	opcode

- Represented as

0000 0001 0101 1010 0000 0100 1011 0011_{two} = 015A04B3₁₆

- Hexadecimal representation is used

Hexadecimal	Binary	Hexadecimal	Binary	Hexadecimal	Binary	Hexadecimal	Binary
0 _{hex}	0000 _{two}	4 _{hex}	0100 _{two}	8 _{hex}	1000 _{two}	c _{hex}	1100 _{two}
1 _{hex}	0001 _{two}	5 _{hex}	0101 _{two}	9 _{hex}	1001 _{two}	d _{hex}	1101 _{two}
2 _{hex}	0010 _{two}	6 _{hex}	0110 _{two}	a _{hex}	1010 _{two}	e _{hex}	1110 _{two}
3 _{hex}	0011 _{two}	7 _{hex}	0111 _{two}	b _{hex}	1011 _{two}	f _{hex}	1111 _{two}

RISC-V I-type format Instructions

■ Problem

- `addi` or `lw` don't require `rs2`. Also, `sw` doesn't require `rd`
- How about to use the `rs2` fields in `addi` for immediates?
A problem occurs that 5 bits is too small to represent some constant

■ Immediate arithmetic and load instructions

- `rs1`: source or base address register number
- immediate: constant operand, or offset added to base address
 - 2s-complement, sign extended

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

■ *Design Principle 3: Good design demands good compromises*

- Different formats complicate decoding, but allow 32-bit instructions uniformly
- Keep formats as similar as possible

RISC-V S-type format Instructions

- Different immediate format for store instructions
 - rs1: base address register number
 - rs2: source operand register number
 - immediate: offset added to base address
 - Split so that rs1 and rs2 fields **always in the same place**

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

RISC-V [R, I, S]-type instruction formats

■ R-type format

- For register

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

■ I-type format

- For arithmetic operands with 1 constant operand
- For load instructions

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

■ S-type format

- For store doubleword instructions

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

RISC-V [R, I, S]-type instruction formats

- Assume the following C code is given

$$A[30] = h + A[30] + 1$$

- x10 has the base of the array A and x21 corresponds to h

- The code is compiled into

- lw x9, 120(x10) # Temporary reg x9 gets A[30]
- add x9, x21, x9 # Temporary reg x9 gets h+A[30]
- addi x9, x9, 1 # Temporary reg x9 gets h+A[30]+1
- sw x9, 120(x10) # Stores h+A[30]+1 back into A[30]

immediate	rs1	funct3	rd	opcode
120	10	2	9	3

funct7	rs2	rs1	funct3	rd	opcode
0	9	21	0	9	51

immediate	rs1	funct3	rd	opcode
1	9	0	9	19

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
3	9	10	2	24	35

Various RISC-V instruction formats

- There are some more instruction formats

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011
Instruction	Format	immediate		rs1	funct3	rd	opcode
addi (add immediate)	I	constant		reg	000	reg	0010011
lw (load word)	I	address		reg	010	reg	0000011
Instruction	Format	immed- iate	rs2	rs1	funct3	immed- iate	opcode
sw (store word)	S	address	reg	reg	010	address	0100011

RISC-V Instructions	Name	Format
Add	add	R
Subtract	sub	R
Add immediate	addi	I
Load word	lw	I
Load word, unsigned	lwu	I
Store word	sw	S
Load halfword	lh	I
Load halfword, unsigned	lhu	I
Store halfword	sh	S
Load byte	lb	I
Load byte, unsigned	lbu	I
Store byte	sb	S
Load reserved	lr.d	R
Store conditional	sc.d	R
Load upper immediate	lui	U
And	and	R
Inclusive or	or	R
Exclusive or	xor	R
And immediate	andi	I
Inclusive or immediate	ori	I
Exclusive or immediate	xori	I
Shift left logical	sll	R
Shift right logical	srl	R
Shift right arithmetic	sra	R
Shift left logical immediate	slli	I
Shift right logical immediate	srl_i	I
Shift right arithmetic immediate	srai	I
Branch if equal	beq	SB
Branch if not equal	bne	SB
Branch if less than	blt	SB
Branch if greater or equal	bge	SB
Branch if less, unsigned	bltu	SB
Branch if greater/eq, unsigned	bgeu	SB
Jump and link	jal	UJ
Jump and link register	jalr	I

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Overall flow from high-level to machine lv.

High-level
language
program
(in C)

```
swap(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

```
swap:
    slli x6, x11, 3
    add  x6, x10, x6
    lw   x5, 0(x6)
    lw   x7, 4(x6)
    sw   x7, 0(x6)
    sw   x5, 4(x6)
    jalr x0, 0(x1)
```

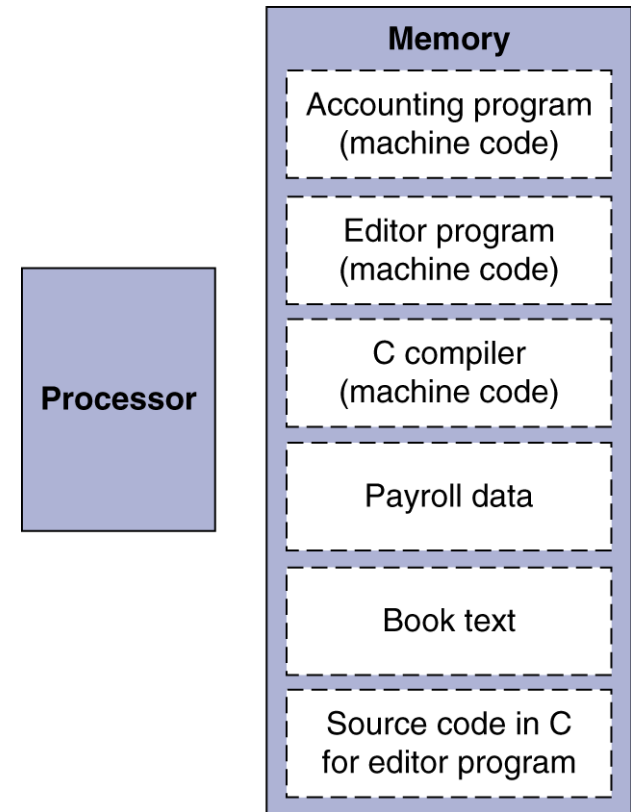
Assembler

Binary machine
language
program
(for RISC-V)

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000001000000001100111
```

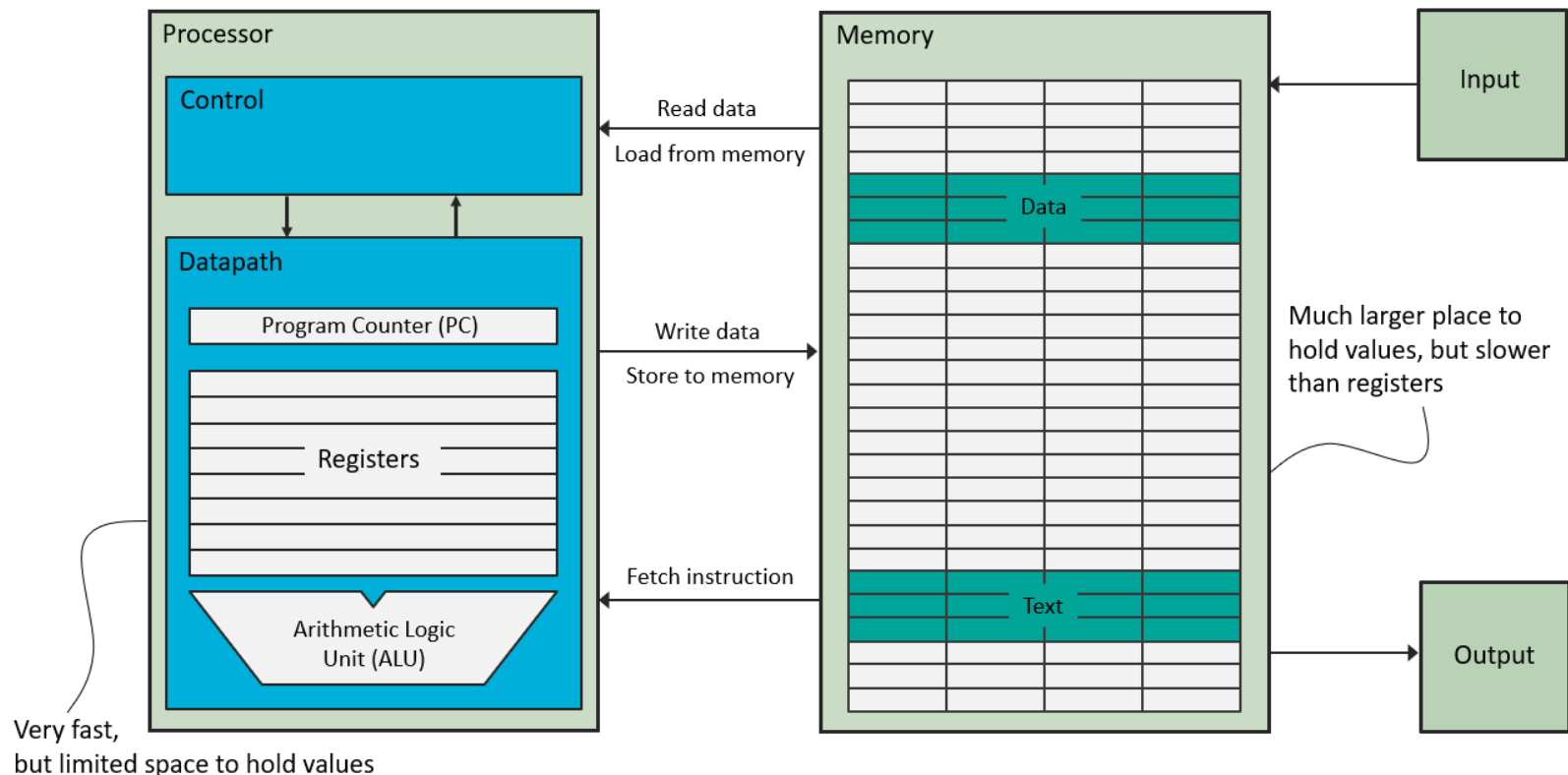
The stored-program concept

- Today's computers are built on two key principles:
 - **Instructions** are represented as binary numbers, **just like data**
 - **Programs** are stored in memory to be read or written, **just like data**
- They lead to **the stored-program concept**; The idea that instructions and data of many types **can be stored in memory as numbers** and thus be easy to change. Its invention let the computing genie out of its bottle!



Program structure

- Programs are stored in memory to be read or written, just like data
- Memory layout** of program:
 - Program (text or code) section: Binary program (instructions) live in
 - Data section: Binary data to perform program

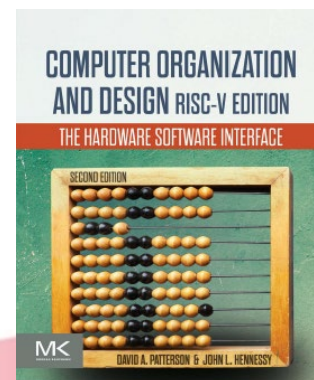


Processor

- Program counter (PC):
 - PC is a register to store the address of **next instruction in program section**
- Processor: The processor interprets (or executes) instructions stored in main memory
 1. The processor **fetches** an instruction, which PC points out, from memory
 2. **Decode**: Figure out which instruction this is
 3. **Execute**: i.e., add numbers, access memory, check a condition, jump to function, and so forth
 4. The processor moves on to the **next instruction** by adding the size of word to PC and so on

Instructions for making decisions

(ch. 2.7, COD book)



Control flow structures in C

1. Sequential (default)
2. Selection / Decision
 - if ... else
 - switch
 - ?: (conditional operator)
3. Repetition / Loop
 - while
 - do ... while
 - For
4. Jump
 - break
 - continue
 - return
 - goto

goto statement in C

- The `goto` statement allows us to **transfer control** of the program to the specified label

```
    goto label;  
    ...  
    ...  
→  label:  
    ...  
    ...
```

- The `label` is an **identifier**. When the `goto` statement is encountered, the control of the program jumps to `label:` and starts executing the code

Conditional branch instruction

- An instruction that tests a value and that allows for a subsequent transfer of control to a new address in the program based on the outcome of the test
- Branch if equal:
 - `if (a == b) goto L1 (in C)`
 - `beq x5, x6, L1 (in RISC-V)`
- Branch if not equal:
 - `if (a != b) goto L1 (in C)`
 - `bne x3, x4, L1 (in RISC-V)`
- Branch if less than:
 - `if (a < b) goto L1 (in C)`
 - `blt x5, x6, L1 (in RISC-V)`
- Branch if greater or equal:
 - `if (a >= b) goto L1 (in C)`
 - `bge x3, x4, L1 (in RISC-V)`

Conditional branch instruction

- `blt`, `bge` consider values in registers as **two's complement number** for comparison
- RISC-V instruction with last character `u` is the instruction for unsigned number
- Branch if less than, unsigned:
 - `if (a < b) goto L1` (in C)
 - `bltu x5, x6, L1` (in RISC-V)
- Branch if greater or equal , unsigned:
 - `if (a >= b) goto L1` (in C)
 - `bgeu x3, x4, L1` (in RISC-V)

Example: if-else statement

- Let a , b , c , d , and e be variables. If the five variables a through e correspond to the five registers $x19$ through $x23$, what is the compiled RISC-V code for this C if statement?

```
if (d == e) a = b + c;  
else a = d - e;
```

- To compile the code segment, we should define a label to make a branch:

```
bne x22, x23, Else    # go to Else if d != e  
add x19, x20, x21     # a = b + c (skipped) if d != e  
beq x0, x0, Exit      # if 0 == 0, go to Exit
```

Else:

```
sub x19, x20, x21     # a = d - e (skipped) if d == e
```

Exit:

Example: while statement

```
while (save[i] == k) i += 1;
```

- Assume that `i` and `k` correspond to registers `x22` and `x24` and the base of the array `save` is in `x25`. Write the RISC-V code corresponding to this C code.

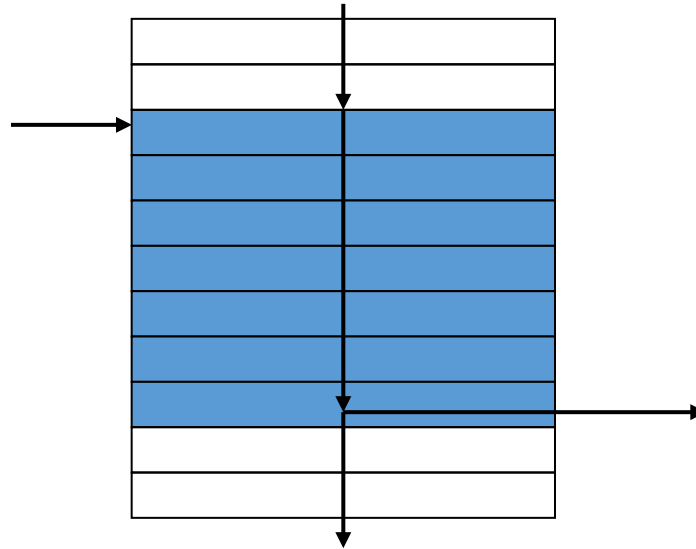
Loop:

```
slli x10, x22, 2    # x10 = i * 4 (offset)
add x10, x10, x25    # x10 = address of save[i]
lw x9, 0(x10)        # x9 = save[i]
bne x9, x24, Exit    # go to Exit if save[i] != k
addi x22, x22, 1     # i = i + 1
beq x0, x0, Loop     # go to Loop
```

Exit: ...

Basic Blocks

- A basic block is a sequence of instructions with
 - No embedded branches (except at end)
 - No branch targets (except at beginning)



- A compiler identifies basic blocks **for optimization**
- An advanced processor can accelerate execution of basic blocks

PC-relative addressing

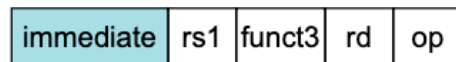
- An addressing regime in which the address is the sum of the program counter (PC) and a constant in the instruction

$$\text{memory address} = \text{PC address} + \text{offset}$$

- We can think PC-relative memory addressing scheme as the special case of base addressing, which base address lives in PC instead of general-purpose register

PC-relative addressing – Cont'd

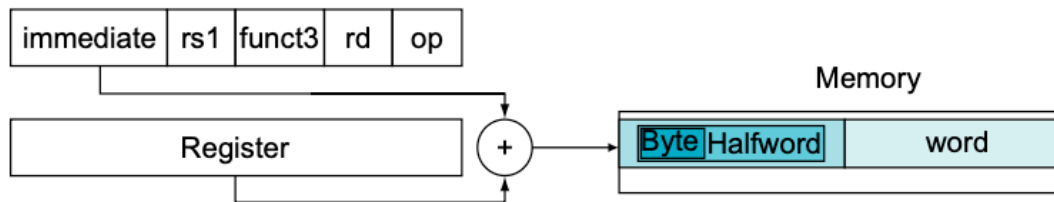
1. Immediate addressing



2. Register addressing



3. Base addressing



4. PC-relative addressing

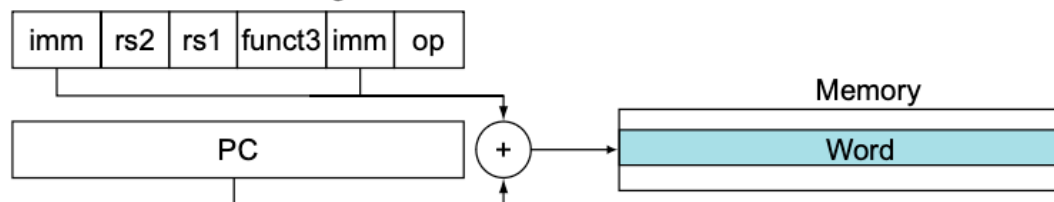


FIGURE 2.17 Illustration of four RISC-V addressing modes. The operands are shaded in color. The operand of mode 3 is in memory, whereas the operand for mode 2 is a register. Note that versions of load and store access bytes, halfwords, words. For mode 1, the operand is part of the instruction itself. Mode 4 addresses instructions in memory, with mode 4 adding a long address to the PC. Note that a single operation can use more than one addressing mode. Add, for example, uses both immediate (addi) and register (add) addressing.

Unconditional branch instruction

- An instruction that allows for a subsequent transfer of control to a new address
- Jump and link:
 - Jump to address and write return address to register
 - `jal x1, 128` (in RISC-V)
- Jump and link with register:
 - Jump to address in register and write return address to register
 - `jalr x0, 0(x1)` (in RISC-V)

Thank you!